Project Report

AIRA Capstone Project: Robotics and Artificial Intelligence

Name:	Marieta Flamouridi
Project Title:	Grid-World Robot with A*, Dijkstra and BFS Path Planning
E-mail:	marieta.flam@gmail.com

1 Project Overview

For this project, a 2D grid-based terrain with various obstacles will be created. On the grid, there will be applied the commonly used pathfinding algorithms A*, Dijkstra and BFS. The goal will be to highlight the best way a hypothetical robot could navigate among obstacles and between a starting and an ending point. There were executed 4 experiments: one without weighted paths, one with weighted paths, one generating 10 random grids and one trying to predict the next move of the robot navigating each of the random grids and according to each of the algorithms, by using Artificial Intelligence.

All these experiments produced metrics about each one of the algorithms, for instance their runtime, the number of nodes they used, the path length and a visual representation of the expected path the robot would follow. For the last case, metrics about the performance of the ML model were more practical.

The main idea for the project was inspired by the AIRA course called "Artificial Intelligence in Robotics". More specifically, the project combines the learning outcomes from units 37 and 51 of the PDF provided within the course, which describe Path Finding Algorithms and Convolutional Neural Networks respectively.

As for the programming language, **Python** was preferred. Alongside Python, some libraries were also included. These were *numpy* for the creation and operation of the 2D grid map, *matplotlib* for the visualisation of the results, *PyTorch* and *scikit-learn* for the AI models implementation.

The whole script is written in a jupyter notebook, along with a few explanations and comments. The screenschots of the results used in the report can also be found in the notebook. The project is uploaded as well on Github here. For the setup instructions, please refer to the README file.

2 Algorithm Descriptions

The following algorithms were chosen because they represent the foundational algorithms used within contemporary real time path planning solutions. Of course, there are many more such algorithms available and research to improve them is still being conducted.

2.1 Breadth First Search (BFS)

The Breadth First Algorithm explores the graph level by level. It starts by visiting all nodes adjacent to the source, moving on then to their adjacent nodes and so on until all reachable nodes are visited. The algorithm indeed guarantees finding the shortest path in terms of number of nodes, however it fails when the edges are weighted. For example, in the picture below, BFS chooses the shortest path A to D even when weights are added on the edges. In this case, the best path should be from A to B to C and to D, so the algorithm fails.

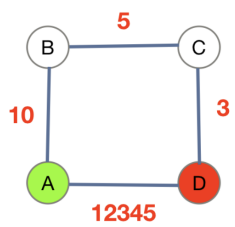

Figure 1: BFS Path Finding with Weighted Edges

2.2 Dijkstra

The Dijkstra's Algorithm solves sub-problems computing the shortest path from the source to vertices among the closest ones to the source. It finds the next closest vertex by maintaining the new vertices in a priority-min queue and stores only one intermediate node, so that only one shortest path is finally found.

Dijkstra's Algorithm is a so-called "greedy" algorithm, as it always chooses the option that gets closer to the solution in the best way given the current situation. This method is virtually identical to BFS, but it requires keeping track of all paths in a priority queue.

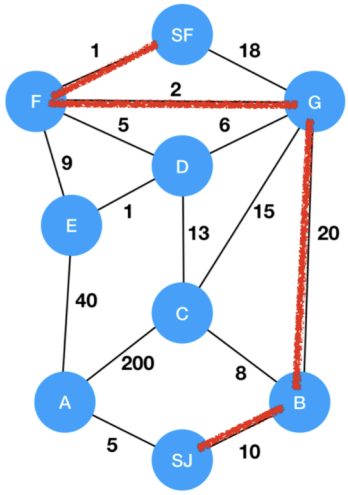

Figure 2: Dijkstra's Algorithm with Weighted Edges

2.3 A*

Since Dijkstra's Algorithm uses only the information provided this specific moment, it ignores sometimes piece of information that might be proven useful to finding the shortest path in the fewest number of steps. The idea of the *heuristic* is then introduced, that is any external useful information about a graph. The A* algorithm adds the heuristic to the value of the priority. In other words:

$$priority(u) = weight(start, u) + heuristic(u, destination), \tag{1}$$

where u is the node under consideration.

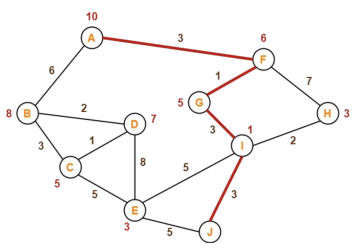

Figure 3: A* Algorithm with Weighted Edges

3 Implementation Details

For each one of the three algorithms as described above, there were executed the following experiments:

1. simple pathfinding case between a starting and an ending point with the same obstacles placed on the 2D grid.

2. weighted pathfinding case between a starting and an ending point with the same obstacles and terrain on the 2D grid. Each type of terrain carries a different weight (water=1, grass=2, land=3).

3. pathfinding between a starting and an ending point with random obstacles generated each time on the grid. In our example, there were used 10 different layouts.

4. predicted pathfinding route between a starting and an ending point using as dataset a total of 50 random layouts as created above using a CNN.

It should be noted that in all cases the robot can move either up, down, right or left, not diagonally.

4 Experimental Results

4.1 Simple case

On the following visual representations, with purple colour is represented the free space where the robot could move, whereas the obstacles are with yellow colour. The path that the robot follows according to each algorithm is green and the starting and ending points are shown as two lighter green dots. All weights are equal to 1 and it is reminded that the movements allowed are only upwards, downwards, left or right.

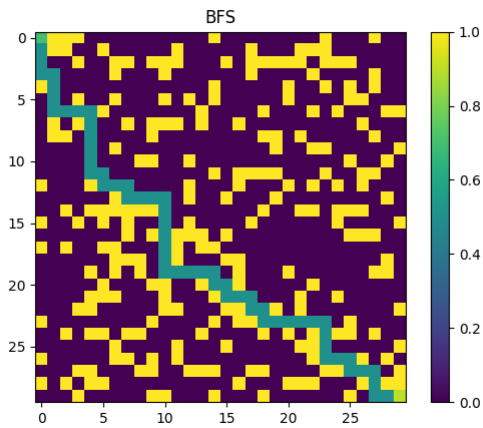

Figure 4: BFS on simple case

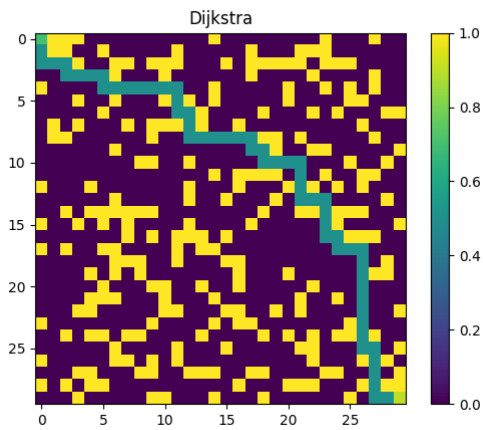

Figure 5: Dijkstra on simple case

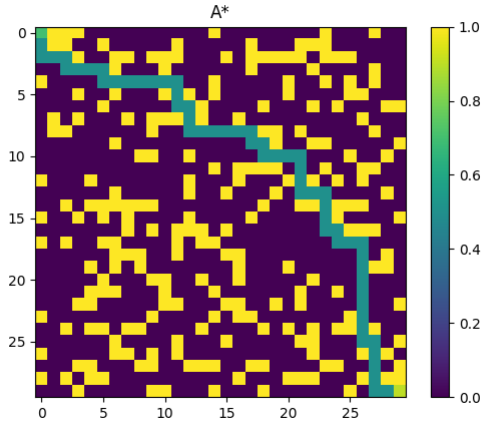

Figure 6: A* on simple case

According to the results provided:

Algorithm	Path length	Nodes Expanded	Runtime
BFS	59	661	0.000639 sec
Dijkstra	59	661	0.002081 sec
A*	59	327	0.001682 sec

As easily deducted, A* algorithm expands fewer nodes and by a significantly low runtime. The path length remains the same, as well as the nodes expanded by BFS and Dijkstra algorithms. BFS is the fastest, however it prefers a different path than the other two algorithms.

4.2 Weighted paths case

On the following representations, the path followed by the robot is red and the starting and ending points can be seen as green and blue dots respectively. There are also some obstacles in black whereas the different weights and terrain colours are as follows:

- the water terrain is blue and equal to 1 in weight scale
- the grass terrain is green and equal to 2 in weight scale
- the land terrain is yellow and equal to 3 in weight scale

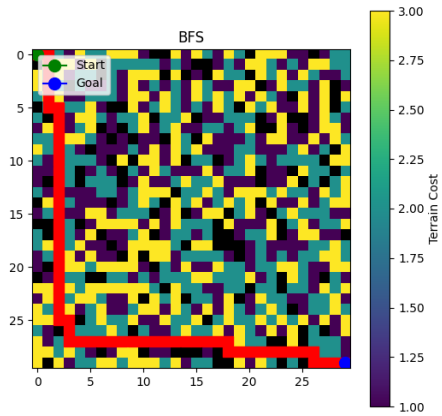

Figure 7: BFS on Weighted Paths case

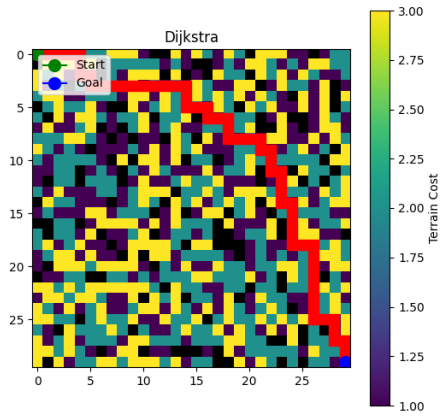

Figure 8: Dijkstra on Weighted Paths case

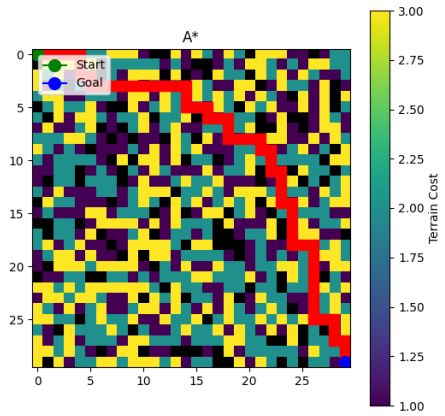

Figure 9: A* on Weighted Paths case

Algorithm	Path length	Nodes Expanded	Runtime
BFS	59	781	0.000678 sec
Dijkstra	59	780	0.001725 sec
A*	59	627	0.002951 sec

As previously, the A* algorithm uses the fewest number of nodes, whereas BFS is still the fastest algorithm, because it does not compute the different weights. That is also obvious by the distinct path it highlights. The path length still remains the same for all algorithms and the nodes expanded by BFS and Dijkstra are equal.

4.3 Random generated grids case

For the generation of random grid layouts, we consider the simple case, without the weighted terrain. The colours are: purple for the free space, yellow for the obstacles, red for the suggested path the robot should follow, blue dot for the starting point and green dot for the ending point. The pictures below show the path all three algorithms suggest for each random grid.

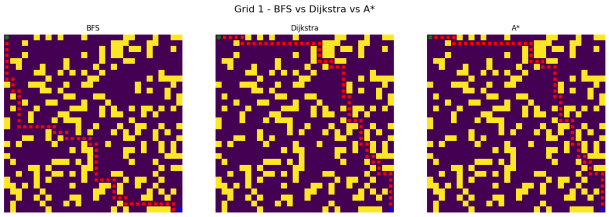

Figure 10: Path Finding for random grid 1

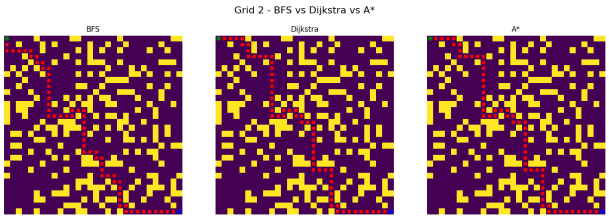

Figure 11: Path Finding for random grid 2

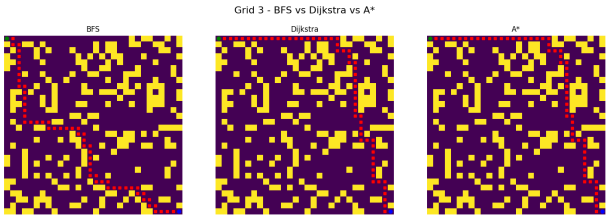

Figure 12: Path Finding for random grid 3

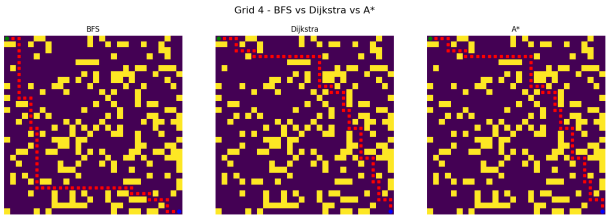

Figure 13: Path Finding for random grid 4

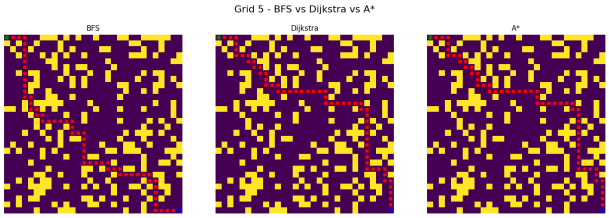

Figure 14: Path Finding for random grid 5

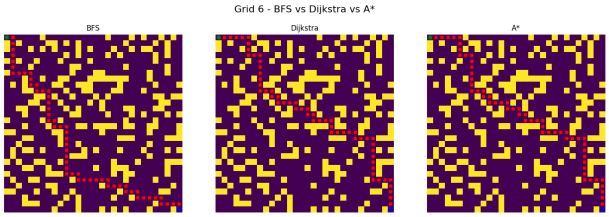

Figure 15: Path Finding for random grid 6

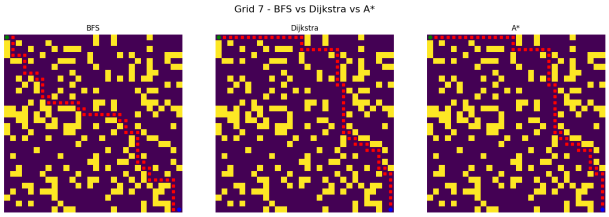

Figure 16: Path Finding for random grid 7

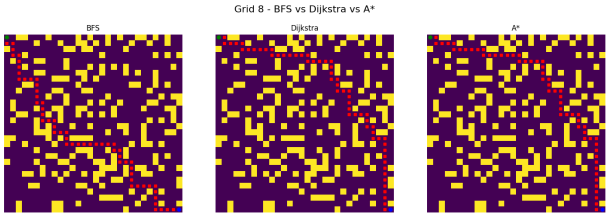

Figure 17: Path Finding for random grid 8

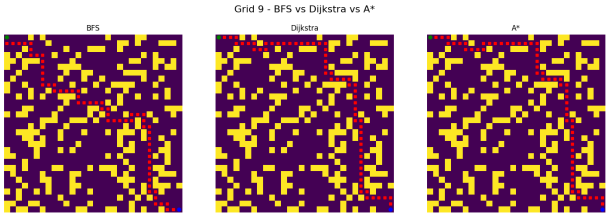

Figure 18: Path Finding for random grid 9

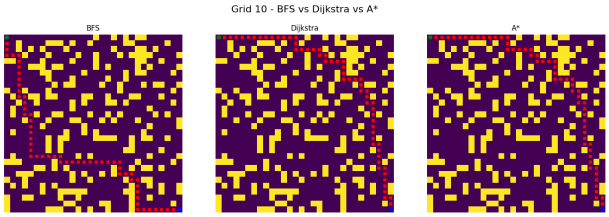

Figure 19: Path Finding for random grid 10

As seen above, in all cases the BFS algorithm selects a different path than the other two. On average over these 10 grids:

Algorithm	**Path length**	**Nodes Expanded**	**Runtime**
BFS	59.40	666.6	0.000573 sec
Dijkstra	59.40	666.6	0.000718 sec
A*	59.40	394.2	0.000499 sec

For that case, althouth all algorithms' path length is the same, A* algorithm is the fastest on average and expands the least number of nodes.

4.4 Path prediction using AI case

For an AI application of the pathfinding algorithms, the previous case is used, which means that the terrain is unweighted and there are only obstacles and free space. There are generated 50 random grids as previously and they are used as an input dataset in order to train the ML model. The model is not given any other instructions on how each algorithm works. The results of 5 path predictions on random generated grids are shown below. The predicted paths of all algorithms are shown on the same grid but with different colours.

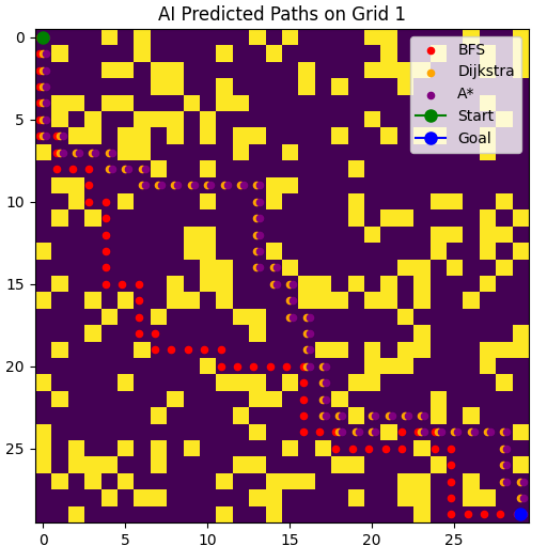

Figure 20: Path prediction on random grid 1

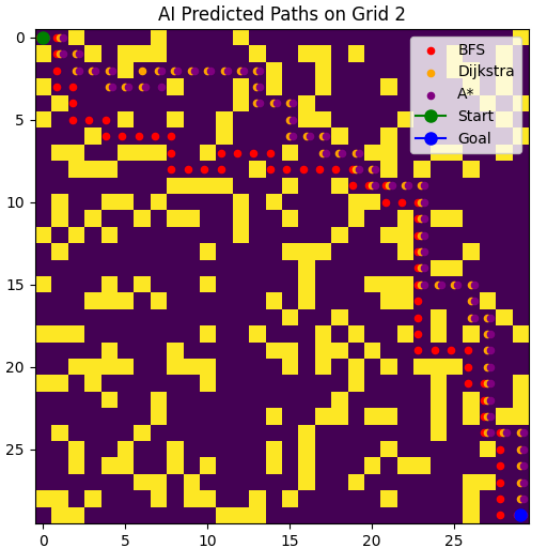

Figure 21: Path prediction on random grid 2

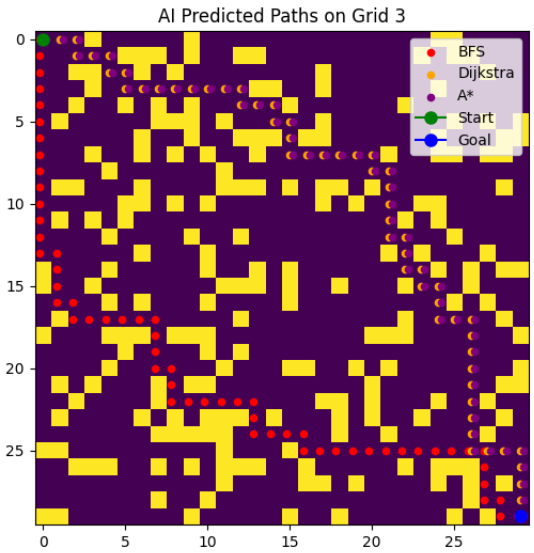

Figure 22: Path prediction on random grid 3

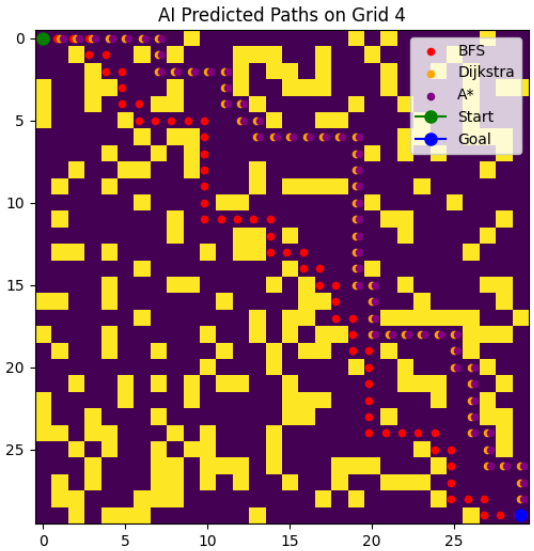

Figure 23: Path prediction on random grid 4

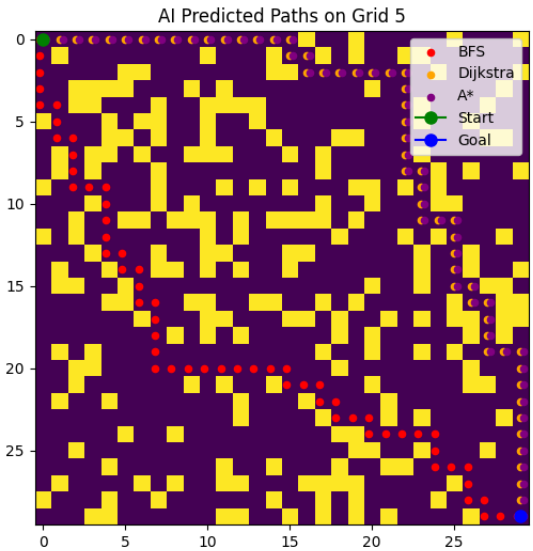

Figure 24: Path prediction on random grid 5

As shown, the paths indicated by A* and Dijkstra algorithms are the same in all cases, whereas BFS chooses another path. On average, for the 50 randomly generated grids it is estimated:

Algorithm	Path length	Nodes Expanded	Runtime
BFS	59.40	667.78	0.000599 sec
Dijkstra	59.40	668.14	0.000747 sec
A*	59.40	354.78	0.000463 sec

A* is estimated to be the fastest algorithm on average and it expands the fewest nodes. The other two algorithms expand a quite similar amount of nodes on average, whereas the path length is the same for all.

Provided that the dataset is bigger, for example 500 random grids, some other metrics can also be used in order to testify the validity of the predicted path. These can be the success rate, the predicted path similarity and the length error. For 500 grids the ML model performance is as follows:

Algorithm	Success Rate	Path Similarity	Length Error
BFS	0.968	0.977	0.958
Dijkstra	0.924	0.9564	2.008
A*	0.972	0.9847	2.168

The results show satisfactory behaviour of the ML model. The model is mostly successful in prredicted A* paths, following by BFS. The same comparison is valid for the path similarity. However, the length calculated resembles mostly with the actual BFS path length.

5 Discussion

5.1 Conclusion

This project aimed to prove understanding of fundamental Artificial Intelligence and Robotics concepts. There were combined three path finding algorithms and an AI model training which predicts the above paths for each algorithm. The algorithms studied were BFS, Dijkstra's and A*. Four distinctive cases were visualized and metrics were provided for the results. In all cases, the A* algorithm seems to have the best performance in terms of the number of nodes expanded and total runtime, whereas Dijkstra's algorithm and BFS present lots of similarities. The CNN model proved a satisfactory performance on predicting the right algorithm path, given 500 different path finding grids. This was primarily due to the small grid (30 x 30) used for this problem, the fixed starting and ending positions and the 4 possible moves the robot could execute (up, down, left, right). If any of these parameters change, the ML model might not be of the same accuracy rate.

5.2 Expansion ideas

There are many short add-ons that can expand the project. For example, the robot could be able to move diagonally as well, the obstacles could move dynamically, the grid could be larger, the starting and ending points could be in random places of the grid each time and so on. All these expansions make more interesting the ML model training as its performance is expected to differ.

References

[1] K. Karur, N. Sharma, C. Dharmatti, and J. e. Shiegel, "A Survey of Path Planning Algorithms for Mobile Robots," *Vehicles*, 2021, Accessed: 2026-11-03.

[2] C. Gregg and Y. Alonso, "Dijkstra's algorithm and the A* algorithm," Fall 2025.

[3] R. Vrabic, "Robotic systems (AIRA edition)," 2025.

[4] M. Goodrich and R. Tamassia, "Algorithm Design and Applications," *Wiley*, 2015.

[5] H.-Y. Hsueh, A.-I. Toma, H. A. Jaafar, E. Stow, R. Murai, P. H. Kelly, and S. Saeedi, "Systematic Comparison of Path Planning Algorithms using PathBench," *Advanced Robotics Journal*, 2022, Accessed: 2026-11-03.

[6] F. Wang, W. Sun, P. Yan, H. Wei, and H. Lu, "Research on Path Planning for Robots with Improved A* Algorithm under Bidirectional JPS Strategy," *Advances in Robot Planning, Volume II*, 2024, Accessed: 2026-11-03.